Ders 3 Genişletilmiş Notu: İlişkilendirme Kuralı Madenciliği ve Apriori Algoritması

Ders 3 Genişletilmiş Notu: İlişkilendirme Kuralı Madenciliği ve Apriori Algoritması

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Konusu ve Planı
- 2. Motivasyon: Çapraz Satış ve Üst Satış
- 3. Market Sepeti Analizi
- 4. Gösterim: Öğe, İşlem ve İşlem Veri Kümesi
- 5. Market Sepetinden Web Madenciliğine
- 6. Kuralın Yapısı ve Öğe Kümeleri
- 7. Destek ve Güven
- 8. Örnek 1: Beş İşlemden Kural Çıkarma
- 9. Örnek 2: En Büyük Sık Öğe Kümesinden Kural Çıkarma
- 10. Ek Nitelikler ve Basitleştirilmiş Bakış
- 11. Apriori Algoritması: Genel Yapı
- 12. Aşağı Kapanış Özelliği
- 13. Düzey Düzey Arama
- 14. Aday Üretme: Birleştirme ve Budama
- 15. Sık Öğe Kümelerinden Kural Üretme
- 16. Pratikte Apriori: k Sınırı ve Maliyet
- 17. Veri Biçimleri
- Örnek Sorular

1. Dersin Konusu ve Planı

Hoca derse, bu haftanın konusunun ders kitabının 2. bölümü olan “Association Rules and Sequential Patterns” olduğunu söyleyerek başladı. Kitabın bölümlerine bağlı kalarak ilerleyeceğini, 2. bölümün slaytlarının ve kitabın PDF’inin ders sayfasından alınabileceğini belirtti.

> **Ek bilgi (derste anlatılmadı):** Bölüm adı ve sıralı örüntü konusunun aynı bölümde yer alması, ders kitabının büyük olasılıkla Bing Liu’nun “Web Data Mining” kitabı olduğunu gösterir. Bu kitapta 2. bölüm ilişkilendirme kuralları ve sıralı örüntüleri birlikte ele alır.

Hocanın planı şöyleydi:

- İlişkilendirme kuralı madenciliği (association rule mining) ve sıralı örüntü madenciliğinin (sequential pattern mining) web madenciliğinde nerelerde kullanıldığı ve motivasyonu.
- İlişkilendirme kuralı madenciliğinde yaygın kullanılan Apriori algoritması.
- Veri biçimleri (data formats).
- İlişkilendirme kuralı madenciliğinin sınıf etiketi olan verilere uygulanması.
- Vakit kalırsa sıralı örüntü madenciliği.

Ders sonunda sınıf etiketli veriler ve sıralı örüntüler kısmına geçilemedi. Hoca sıralı örüntü madenciliğinin de en az ilk kısım kadar önemli olduğunu, bu yüzden ona yeterli zaman ayırmak için gelecek haftaya bıraktığını söyledi.

2. Motivasyon: Çapraz Satış ve Üst Satış

Hoca konuya günlük hayattan bir gözlemle girdi. Yakınımızdaki bir süpermarkete ya da bir e-ticaret sitesine gittiğimizde bazı ürünlerin bir paket hâlinde, indirimli fiyatla birlikte satıldığını görürüz. Hocanın sorusu şuydu: Bu şirketler hangi ürünleri bir araya koyacaklarına nasıl karar veriyor?

2.1 Çapraz Satış (Cross-sell)

Pazarlamada bu tür satışa çapraz satış (cross-sell) denir. Çok satılan, popüler bir ürün seçilir. Yanına o kadar popüler olmayan ikinci bir ürün konur. İkisi bir paket yapılıp satılır. Müşteri asıl popüler ürünü almak ister. Paketteki ikinci ürünün orada olup olmadığını pek umursamaz. Böylece az satılan ürün de satılmış olur.

2.2 Üst Satış (Upsell)

İkinci kavram üst satıştır (upsell). Özellikle hizmet sektöründe kullanılır. Ürün yerine hizmet satın aldığınızı düşünün. Bir hizmete belli bir seviyede, örneğin normal üyelikle abone olursunuz. Hizmet veren şirketler zaman zaman yeni pazarlama fikirleri geliştirir. Müşteriyi kullandığı seviyenin bir üstüne, uygun bir fiyatla geçirmeye çalışırlar.

Bunun için önce hangi hizmetlerin hangi üyelik türlerinin müşteriler arasında popüler olduğunu bilmek gerekir. Bu bilinirse, o üyelikler daha üst bir seviyeye yükseltilip makul bir fiyatla satışa sunulabilir.

2.3 Bilişimcinin Rolü

Pazarlamacıların bu iki stratejideki asıl amacı şirketin kârını artırmaktır. Bunun için bilişimcilerden, müşterilere birlikte ve sık satılan popüler ürünleri bulmalarını beklerler. Bu bilgi elde edilirse çapraz satış ve üst satış stratejileri buna göre kurulur. Bilgisayar bilimi ve veri bilimi de veriyi analiz edip birlikte satılan popüler ürünleri bulan algoritmalar üzerinde çalışır.

3. Market Sepeti Analizi

Hocaya göre bu kategorideki ilk algoritma 1993'te Agrawal adlı bir araştırmacı tarafından önerildi. Agrawal algoritmayı tanıtırken problemi market sepeti analizi (market basket analysis) problemi diye adlandırdı.

Problemin arkasındaki fikir şudur: Müşterilerin birlikte satın aldığı öğeler ilişkilidir. Çok sayıda müşteri bazı ürünleri birlikte alıyorsa, bu ürünler birbiriyle ilişkilidir. İlişkili ürünler bulunursa, mağazada birbirine yakın yerlere konabilir ya da çapraz satış veya üst satışa konu olabilir.

Bu algoritmaların çıktısı bir kuraldır. Slayttaki örnek kural:

$$\text{Bread} \rightarrow \text{Milk} \quad [\text{sup} = \%5,\ \text{conf} = \%100]$$

Kural, "ekmek alan kişinin süt de alması olasıdır" demektir. Pazarlamacılar böyle bir kuralı çapraz satış çalışması için başlangıç noktası olarak kullanabilir. Algoritmalar kuralı bulurken kuralın kalitesini gösteren ölçüler de verir. Burada iki ölçü vardır:

- **Destek (support):** Tüm işlemlerin içinde, hem ekmek hem süt içeren işlemlerin oranı. Buradaki %5, tüm işlemlerin %5'inde ekmek ve sütün birlikte bulunduğunu söyler. Başka bir deyişle tüm işlemlerin %5'i bu kuralı "destekler". Adı buradan gelir.
- **Güven (confidence):** "Bu kurala ne kadar güvenebilirim?" sorusunun cevabı. Hesabı farklıdır: Ekmek içeren işlemlerin kaçında süt de vardır?

Hoca, öğreneceğimiz algoritmaların ilişkilendirme kurallarını üretirken bu destek ve güven değerlerini kullandığını vurguladı.

4. Gösterim: Öğe, İşlem ve İşlem Veri Kümesi

Algoritmaya geçmeden önce market sepeti probleminin gösterimi tanıtıldı.

- **Öğeler kümesi:** $I = \{i_1, i_2, \ldots, i_m\}$. İşlem veritabanındaki, yani mağazadaki tüm öğelerdir. Fiziksel bir süpermarket düşünürseniz, o mağazada satılan bütün ürünler I'nın içindedir.
- **İşlemler:** $T = \{t_1, t_2, \ldots, t_n\}$. Mağazada gerçekleşen bütün işlemlerdir. Her işlem t_i, bir öğe kümesidir ($t_i \subseteq I$).

Hoca işlemi (transaction) kasa fişiyle anlattı. Bir mağazada ürünleri seçip kasaya gelirsiniz, ödemeyi yaparsınız ve karşılığında bir fiş alırsınız. Fişte o alışverişte aldığınız bütün öğelerin listesi vardır. Ödeme yaptığınız o tek etkinlikte satın alınan ürünlerin hepsi bir işlemdir. Aynı gün bir markete üç kez gidip üç kez ödeme yaparsanız üç ayrı işlem oluşturmuş olursunuz.

Her işlemin bir kimliği (transaction ID, TID) vardır. İşlemlerdeki öğe sayıları farklı olabilir. Bütün işlemler bir araya getirildiğinde işlem veri kümesi (transactional data set) oluşur.

Çevrimiçi ya da fiziksel perakende mağazaları işlem verilerini günlük olarak tutar. Günlük işlemler merkezi bir yere gönderilir ve orada hep birlikte işlenir. İşlem sayısı arttıkça veri kümesindeki kayıt sayısı da artar. Sonunda bu, büyük ölçekli bir veriye dönüşür.

5. Market Sepetinden Web Madenciliğine

Problem ilk ortaya çıktığında, yani 1993'te, fiziksel perakende mağazaları vardı. İnsanlar bu mağazaların işlem verilerini işleyip birlikte satılan popüler ürünleri nasıl bulabileceklerini düşünüyordu. Sonra web yaygınlaştı ve e-ticaret siteleri büyük ilgi gördü. O güne kadar geliştirilen algoritmalar e-ticaret sitelerinin işlem verileri üzerinde de kullanılmaya başlandı.

Bir noktada araştırmacılar market sepeti analiziyle web veri madenciliği, yani webdeki belgelerin analizi arasında bir benzerlik kurabileceklerini fark etti. Problem şudur: Bir belge kümesinde en çok hangi konulardan ya da hangi anahtar sözcüklerden söz ediliyor?

Slayttaki küçük örnekte yedi belge vardı. Soru, bu belgelerin çoğunda geçen popüler anahtar sözcükleri bulmaktı. Hoca bunun aynı problem olduğunu gösterdi:

Market sepeti analizi	Web belge analizi
İşlem (transaction)	Belge (document)
Öğe (item)	Anahtar sözcük (keyword)
Birçok işlemde birlikte geçen öğeler	Birçok belgede birlikte geçen anahtar sözcükler

Böylece aynı algoritmalar bu problemde de kullanılmaya başlandı. Belgelerin çoğunda geçen anahtar sözcükler bulunursa, "bu belge kümesi özellikle şu anahtar sözcükler üzerine yoğunlaşıyor" denebilir.

6. Kuralın Yapısı ve Öğe Kümeleri

İlişkilendirme kuralı madenciliği algoritmalarının sonunda ürettiği şey kurallardır. Bir kural şu biçimdedir:

$$X \to Y, \qquad X, Y \subset I, \qquad X \cap Y = \emptyset$$

- Kuralın sol tarafında X, sağ tarafında Y vardır. İkisi de birer öğe kümesidir (itemset). Tek öğeden de oluşabilirler, birden fazla öğeden de.
- X ve Y, mağazadaki tüm öğeleri içeren I'nın alt kümeleridir.
- X ile Y aynı öğeyi içermez, yani kesişimleri boş kümedir.

- Bir işlem t, X'in bütün öğelerini içeriyorsa "t, X'i içerir" denir.

Öğe kümeleri, içerdikleri öğe sayısıyla adlandırılır. Üç öğeli bir küme 3-öğe kümesi (3-itemset), tek öğeli bir küme 1-öğe kümesidir (1-itemset). Genel olarak k öğeli kümeye k-öğe kümesi (k-itemset) denir.

7. Destek ve Güven

Hoca daha önce sözlü anlattığı iki ölçüyü bu kez formülle verdi. n toplam işlem sayısıdır.

$$\text{support}(X \to Y) = \frac{(X \cup Y)\text{'yi içeren işlem sayısı}}{n}$$

$$\text{confidence}(X \to Y) = \frac{(X \cup Y)\text{'yi içeren işlem sayısı}}{X\text{'i içeren işlem sayısı}}$$

Destek, hem X'in hem Y'nin öğelerini birlikte içeren işlemlerin tüm işlemlere oranıdır. Hocanın hatırlatma yolu: "İşlemlerin yüzde kaçı bu kuralı destekliyor?"

Güven, kurala ne kadar güvenebileceğimizi söyler. Önce X'i içeren işlemler sayılır. Sonra bunların içinden Y'yi de içerenler sayılır. İkinci sayı birinciye bölünür. Paydadaki sayı yalnız Y'siz X'leri değil, X'i içeren bütün işlemleri kapsar.

> **Ek bilgi (derste anlatılmadı):** Güven, koşullu olasılık (conditional probability) olarak $P(Y \mid X)$ diye de yazılır: işlemin X'i içerdiği bilindiğinde Y'yi de içerme olasılığı. Bu nedenle $\text{confidence}(X \to Y) = \text{support}(X \cup Y)/\text{support}(X)$ olur; paydaki ve paydadaki n sadeleşir.

Soru: Destek formülündeki ifadenin birleşim değil de kesişim olması gerekmiyor mu?

Cevap: Hayır. Kuralın başında X ile Y'nin kesişiminin boş küme olduğunu zaten söylemiştik. X birden fazla öğeden, Y de birden fazla öğeden oluşabilir, ama aynı öğeleri içermezler. Desteği hesaplarken, iki taraftaki öğelerin hepsini birlikte içeren işlemleri buluruz. Bu öğelerin kümesi $X \cup Y$'dir. Bu işlemlerin sayısı paya, toplam işlem sayısı paydaya yazılır.

7.1 Problemin Amacı

İlişkilendirme kuralı madenciliğinin amacı, hem en küçük destek (minimum support, minsup) hem en küçük güven (minimum confidence, minconf) eşiğini geçen kuralları bulmaktır. Hangi algoritmayla çalışırsanız çalışın, sizden bu iki eşik değerini ister.

Bu algoritmaların kalitesi şöyle tanımlanır: Veride bulunan ve destek ile güveni eşiklerin üzerinde olan bütün kuralları bulmaya çalışırlar. Kuralın sağ tarafında bir hedef ya da sınıf etiketi (label) yoktur. Kuralın iki tarafında da yalnızca öğeler bulunur.

8. Örnek 1: Beş İşlemden Kural Çıkarma

Hoca ilk örnekte beş işlem verdi. Eşikler minsup = %50, minconf = %50'ydi. Soru: Bu veri kümesinden ilişkilendirme kuralları üretebilir misiniz?

TID	Alınan öğeler
10	A, B, D
20	A, C, D
30	A, D, E
40	B, E, F
50	B, C, D, E, F

1. adım: Tek öğeler. Hoca önce tek öğelerin kaç işlemde geçtiğini saydı.

- A, 3 işlemde geçer: $3/5 = \%60 \geq \%50$, yazılır.

- B, 3 işlemde geçer: $3/5 = \%60$, yazılır.
- D, 4 işlemde geçer: $4/5 = \%80$, yazılır.
- E, 3 işlemde geçer: $3/5 = \%60$, yazılır.
- C ve F yalnız 2 işlemde geçer: $2/5 = \%40 < \%50$, elenir (ara adım eklendi).

2. adım: İki öğeli kümeler. AD, 3 işlemde birlikte geçer (10, 20, 30): $3/5 = \%60$, yazılır. Hoca burada devam edilebileceğini ama durduğunu söyledi. Böylece sık örüntüler (frequent patterns) şunlardır:

$$\{A{:}3,\ B{:}3,\ D{:}4,\ E{:}3,\ AD{:}3\}$$

3. adım: Kurallar. Tek öğeli kümelerle bir şey yapılamaz, çünkü her kuralın bir sol bir sağ tarafı vardır. Kural yazmak için en az iki öğeli bir küme gerekir. Burada yalnız $\{A, D\}$ vardır. İki olası kural vardır: A solda D sağda ya da D solda A sağda.

İki kuralın destek değeri aynıdır. Çünkü destek, hem A hem D içeren işlem sayısının toplam işlem sayısına oranıdır ve bu iki kural için de aynıdır:

$$\text{support}(A \to D) = \text{support}(D \to A) = \frac{3}{5} = \%60$$

Güven değerleri ise farklıdır:

- $A \to D$: A içeren işlem sayısı 3, paydaya yazılır. Hem A hem D içeren işlem sayısı 3. Güven $3/3 = \%100$.
- $D \to A$: D içeren işlem sayısı 4, paydaya yazılır. Hem D hem A içeren işlem sayısı 3. Güven $3/4 = \%75$.

İki kuralın güveni de %50'nin üzerinde olduğu için ikisi de sonuç kural kümesine girer. Hoca, bu problemde izlediği yolun Apriori algoritmasının izlediği yolla aynı olduğunu söyledi: önce sık öğe kümeleri, sonra iki ve daha çok öğeli her küme için bütün olası kurallar, sonra bunların güven hesabı.

Python ile sayımlar kontrol edildi; yukarıdaki değerlerin hepsi doğrulandı.

Hoca bunun çok az işlem ve öğe içeren bir oyuncak örnek (toy example) olduğunu hatırlattı. Gerçek işlem veri kümelerinde milyonlarca işlem ve binlerce ürün olur. Amazon gibi büyük e-ticaret sitelerinde milyarlarca işlem ve milyonlarca ürün vardır. Bu yüzden hem çok sayıda işleme ölçeklenebilen hem de veride bulunan kuralları bulacak kadar doğru çalışan algoritmalara ihtiyaç vardır.

Soru: Destek neden %60 çıkıyor? Formülde X ile Y'nin birleşimi yazıyor; o zaman A veya D içeren işlemleri sayarsak %80 çıkması gerekmez mi?

Cevap: Destek, kuralın sol ve sağ tarafındaki öğelerin hepsini birlikte içeren işlemlerin toplam işlem sayısına oranıdır. "A varsa D de vardır" kuralında sol tarafta A, sağ tarafta D var. İki taraftaki öğeleri birlikte içeren işlem sayısı 3, toplam işlem sayısı 5. Destek $3/5$. Buradaki birleşim, "A ya da D'yi içeren" anlamında değil, "her iki tarafın bütün öğelerini içeren" anlamındadır.

9. Örnek 2: En Büyük Sık Öğe Kümesinden Kural Çıkarma

İkinci örnekte yedi işlem vardı. Eşikler minsup = %30, minconf = %80'di.

TID	Öğeler
t1	Beef, Chicken, Milk
t2	Beef, Cheese
t3	Cheese, Boots
t4	Beef, Chicken, Cheese
t5	Beef, Chicken, Clothes, Cheese, Milk
t6	Chicken, Clothes, Milk

TID	Öğeler
t7	Chicken, Milk, Clothes

Hoca bu örnekte sık öğe kümelerinin bir algoritma tarafından zaten bulunduğunu varsaydı. Algoritma en çok öğe içeren sık öğe kümesini getirmişti:

$$\{\text{Chicken, Clothes, Milk}\}, \qquad \text{support} = \frac{3}{7}$$

Bu kümenin sık olmasının sebebi, desteğinin verilen en küçük destek değerinin üzerinde olmasıdır: yedi işlemin üçü (t5, t6, t7) bu üç öğeyi birlikte içerir ve $3/7 \approx \%42{,}9 \geq \%30$.

Hoca, bu kümenin alt kümelerinin de (örneğin $\{\text{Chicken, Clothes}\}$ ya da $\{\text{Clothes, Milk}\}$) sık olduğunu söyleyebileceğimizi belirtti. Ama bu örnekte yalnız en büyük sık öğe kümesiyle ilgilendi.

Kural yazma yöntemi: Önce çıkarılabilecek bütün kurallar yazılır. Clothes sola, kalanlar sağa. Milk sola, kalanlar sağa. Chicken sola, kalanlar sağa. Sonra iki öğeli sol taraflara geçilir: Clothes ve Chicken sola, kalan sağa, vb. Sonra her kuralın güveni hesaplanır ve güveni en küçük güven değerinin altında kalanlar elenir.

Kümeden çıkan bütün kuralların destek değeri $3/7$'dir. Güvenler (Clothes için hocanın hesabı: Clothes içeren işlem 3, üç öğeyi birlikte içeren işlem 3, güven $3/3 = \%100$):

Kural	Sol tarafı içeren işlem	Güven	minconf %80
Clothes → Milk, Chicken	3	$3/3 = \%100$	geçer
Clothes, Chicken → Milk	3	$3/3 = \%100$	geçer
Clothes, Milk → Chicken	3	$3/3 = \%100$	geçer
Milk → Chicken, Clothes	4	$3/4 = \%75$	geçmez
Chicken, Milk → Clothes	4	$3/4 = \%75$	geçmez
Chicken → Clothes, Milk	5	$3/5 = \%60$	geçmez

Python ile en büyük sık öğe kümesinin $\{\text{Chicken, Clothes, Milk}\}$ olduğu ve tablodaki güvenler doğrulandı.

Hoca iki örneği karşılaştırdı. İlk örnekte önce tek öğeler, sonra iki öğeli kümeler sayıldı, sık örüntüler bulundu, en büyük sık öğe kümesinden bütün kurallar yazıldı. İkinci örnekte de en büyük sık öğe kümesinden başlanıp bütün olası kurallar yazıldı, her birinin güveni hesaplandı ve güveni eşiğin üzerinde olanlar kullanılabilir kurallar olarak alındı.

Soru: Chicken ve Milk sol tarafta, Clothes sağ tarafta olsaydı sonuç farklı mı olurdu?

Cevap: Hoca kuralı tahtaya yazıp hesapladı. Önce Milk ve Chicken'ın birlikte geçtiği işlemlere bakılır: dört işlem (t1, t5, t6, t7). Bunların içinden iki tarafın bütün öğelerini içerenler üç tanedir. Güven $3/4 = \%75$ çıkar. Kuralı isteyen kişi (pazarlamadaki kişi) en küçük güveni %80 olarak belirlediği için bu kural alınamaz.

Hoca bunun anlamını da açıkladı. Güven, "Birisi Milk ve Chicken almışsa, veriye bakarak bu kişinin Clothes da alacağını hangi güvenilirlikle bilebilirim?" sorusunun cevabıdır. Clothes → Milk, Chicken kuralında veri şunu söyler: Daha önce gerçekleşmiş bütün durumlarda Clothes alan herkes Milk ve Chicken da almıştır, yani %100. Milk ve Chicken alanların ise ancak %75'i Clothes da almıştır. Kuralı isteyen kişi ise "Bana öyle kurallar bul ki geçmişte gerçekleşen olaylara bakıldığında en az %80'inde bu kural işlemiş olsun" demiştir.

> **Hocanın vurgusu:** Pratikte kuralların güven değerleri genellikle yüksek, destek değerleri ise düşük olur. Olabildiğince çok ve farklı sık öğe kümesi bulunması istenir; ama bunların içinden çıkan kuralların, gerçekleşen durumlar içinde en yüksek güvenilirlikte olması beklenir.

10. Ek Nitelikler ve Basitleştirilmiş Bakış

Hoca, işlem veri kümelerinde genellikle şimdiye kadar gösterilenden daha fazla bilgi olduğunu belirtti. Örneklerde her işlem için yalnız işlem kimliği ve öğeyi gösteren bir sembol vardı. Gerçek veride ise miktar (quantity), fiyat (price) ve öğeyle ilişkili başka nitelikler de bulunur. İlişkilendirme kuralı madenciliği algoritmalarının yeni sürümleri bu ek nitelikleri de hesaba katar.

Bu derste temeller öğrenildiği için problem, yalnız işlem kimliği ve öğe kimlikleri varmış gibi ele alınır. Buna alışveriş sepetinin basitleştirilmiş bakışı (simplistic view of the shopping basket) denir.

Hoca problemin çok popüler olduğunu da ekledi. Fiziksel ya da çevrimiçi her perakende mağazası bu problem üzerinde günlük olarak çalışır, farklı ilişkilendirme kuralı madenciliği algoritmaları çalıştırır. Bu problemi çözmek için özel olarak yazılmış hem kapalı kaynak hem açık kaynak yazılım paketleri vardır.

11. Apriori Algoritması: Genel Yapı

Aradan sonra hoca Apriori algoritmasının ayrıntılarına geçti.

İşlem veri kümesi T, en küçük destek ve en küçük güven verildiğinde, T'de bulunan ilişkilendirme kuralları kümesi tek türlü belirlenir. Apriori algoritması da bunu yapar: Girdi olarak T'yi, minsup ve minconf değerlerini alır. Çıktı olarak destek ve güven koşullarını sağlayan ilişkilendirme kurallarını üretir.

Algoritma veri madenciliğinde, özellikle ilişkilendirme kuralı madenciliğinde, çok bilinen bir algoritmadır ve iki adımdan oluşur:

1. **Sık öğe kümelerini bul:** Desteği en küçük destek değerinin üzerinde olan bütün öğe kümelerini (frequent itemsets) bul.
2. **Kural üret:** Bulunan sık öğe kümelerini kullanarak kuralları üret.

Hoca bunu bir önceki örnekle bağladı: Örnek 2'de üç öğeli sık öğe kümesi bulunmuş, sonra bütün olası kurallar yazılmış ve güveni düşük olanlar elenmişti.

12. Aşağı Kapanış Özelliği

Apriori'ye göre bir öğe kümesinin sık öğe kümesi olabilmesi için Apriori özelliğini, diğer adıyla aşağı kapanış özelliğini (downward closure property) sağlaması gerekir: Bir öğe kümesi sıksa, onun bütün alt kümeleri de sıktır.

Hocanın örneği $\{A, B, D\}$ kümesiydi. Bu kümenin sık olduğunu söylüyorsak iki koşulu sağlamalıdır:

- **Birinci koşul:** Desteği en küçük destek değerinin üzerinde olmalıdır.
- **İkinci koşul:** Bütün alt kümeleri de sık olmalıdır, yani her alt kümenin desteği de eşiğin üzerinde olmalıdır.

Bu özelliğin tersten okunuşu algoritmada asıl işe yarayan kısımdır: Bir kümenin alt kümelerinden biri sık değilse, kümenin kendisi de sık olamaz. Bu, aday kümelerin veriye bakmadan elenmesini sağlar (Bölüm 14).

13. Düzey Düzey Arama

13.1 Yinelemeli, Aşağıdan Yukarıya

Apriori yinelemeli (iterative) bir algoritmadır, yani bir döngü içinde çalışır. Düzey düzey arama (level-wise search) yapar ve aşağıdan yukarıya (bottom-up) ilerler:

1. Önce 1-öğe kümelerini sayar, sık olanları bulur.
2. Sık 1-öğe kümelerini kullanarak sık olabilecek 2-öğe kümelerini, yani adayları üretir.
3. Bu adayların gerçekten sık olup olmadığını kontrol eder ve gerçek sık 2-öğe kümelerini bulur.

4. Gerçek sık 2-öğe kümelerini kullanarak olası 3-öğe kümelerini üretir, bunların gerçekten sık olup olmadığını kontrol eder.
5. Bu böyle devam eder.

13.2 Algoritmanın Akışı

Hoca işlem veri kümesini milyonlarca satırı olan büyük bir tablo gibi düşünmemizi istedi. Her satır bir işlemdir ve satırda o işlemde alınan öğelerin listesi vardır.

Algoritma başta bütün işlemleri okuyan bir geçiş (pass) yapar. Her öğenin kaç işlemde göründüğünü sayar. Sayısı en küçük destek eşiğinin altında kalan öğeleri eler. Kalan öğelere F_1 denir: sık 1-öğe kümeleri.

Gösterimde iki harf kullanılır:

- F_k: gerçek sık k-öğe kümeleri (frequent). Desteği hesaplanmış ve eşiği geçtiği bilinen kümeler.
- C_k: aday k-öğe kümeleri (candidate). Üretilmiş ama gerçekten sık olup olmadıkları henüz bilinmeyen kümeler.

Sonra algoritma döngüye girer. Döngü $k = 2$'den başlar. Hoca tartışma için $k = 10$'a kadar gidebileceğini örnek verdi. Her turda:

- F_{k-1}'den bütün olası aday k-öğe kümeleri C_k üretilir. Örneğin A, B ve D sık 1-öğe kümeleriyse bunlardan bütün olası 2-öğe kümeleri üretilir ve bu satırın tamamı aday 2-öğe kümeleridir.
- Veritabanı üzerinde bir geçiş daha yapılır. Her adayın kaç işlemde birlikte geçtiği sayılır.
- Geçiş bitince hangilerinin gerçekten sık olduğu bilinir. Slayttaki şemada AB, AD ve BD'nin eşiğin üzerinde birlikte geçtiği bulunmuştu.
- Döngünün başına dönülür, k bir artırılır.

Slayttaki şemada 3-öğe düzeyinde ABC, ABD, ACD gibi dört olası aday vardı ve bunlardan yalnız biri sık çıktı. Algoritma bu şekilde $k = 4$'ten $k = 10$'a kadar devam eder. Amaç, en çok öğe içeren sık öğe kümelerini bulmaktır.

13.3 Sayısal Örnek: Dört İşlem

Hoca algoritmayı dört işlemlik bir oyuncak örnekle gösterdi. En küçük destek %50'dir. Dört işlemde %50, bir öğe kümesinin sık sayılması için **en az 2 işlemde** geçmesi demektir.

TID	Öğeler
T100	1, 3, 4
T200	2, 3, 5
T300	1, 2, 3, 5
T400	2, 5

1. tarama (döngüden önce). Bütün 1-öğe kümeleri sayılır:

$$C_1 : \ \{1\}{:}2,\ \{2\}{:}3,\ \{3\}{:}3,\ \{4\}{:}1,\ \{5\}{:}3$$

$\{4\}$ yalnız bir işlemde geçtiği için elenir:

$$F_1 = \{\{1\}, \{2\}, \{3\}, \{5\}\}$$

$k = 2$**.** F_1'den bütün olası aday 2-öğe kümeleri üretilir. Bunların gerçekten sık olup olmadığı henüz bilinmez:

$$C_2 = \{\{1,2\}, \{1,3\}, \{1,5\}, \{2,3\}, \{2,5\}, \{3,5\}\}$$

2. tarama. Veri yeniden okunur ve her adayın iki öğesini birlikte içeren işlem sayısı yazılır:

$$\{1,2\}{:}1,\ \{1,3\}{:}2,\ \{1,5\}{:}1,\ \{2,3\}{:}2,\ \{2,5\}{:}3,\ \{3,5\}{:}2$$

Eşiğin altındakiler elenir:

$$F_2 = \{\{1,3\},\{2,3\},\{2,5\},\{3,5\}\}$$

$k = 3$**.** Döngünün başına dönülür ve olası 3-öğe adayları üretilir. Slaytta tek aday görünür:

$$C_3 = \{\{2,3,5\}\}$$

3. tarama. $\{2,3,5\}$, T200 ve T300'de geçer, sayısı 2'dir:

$$F_3 = \{\{2,3,5\}\}$$

Python ile her adımın sayımları doğrulandı.

13.4 Neden C_3'te Tek Aday Var?

Hoca burada bir noktaya dikkat çekti. F_2'deki 2-öğe kümeleri birleştirilerek adaylar üretilseydi, C_3'te daha fazla küme görmeyi beklerdik. Örneğin $\{1,2,3\}$ ya da $\{1,3,5\}$ neden yok?

Cevap Apriori'nin iki koşulundadır. Birinci koşulu, yani desteğin eşiğin üzerinde olup olmadığını henüz bilemeyiz, çünkü üçüncü taramayı yapmadık. Ama ikinci koşulu, aşağı kapanış özelliğini, veriye bakmadan kontrol edebiliriz: Adayın bütün alt kümeleri sık olmalıdır.

- $\{2,3,5\}$: $\{2,3\}$ sık, $\{2,5\}$ sık, $\{3,5\}$ sık. İkinci koşulu sağlar, aday olarak kalır.
- $\{1,2,3\}$: $\{1,2\}$ F_2'de yok. İkinci koşulu sağlamaz, atılır.
- $\{1,3,5\}$: $\{1,3\}$ var, $\{3,5\}$ var, ama $\{1,5\}$ yok. Atılır.

Bunun kazancı şudur: Üçüncü taramada daha az sayıda 3-öğe kümesi sayılır. Yalnız ikinci koşulu sağlayan adaylar sayılır, sağlamayanlar için sayım yapılmaz.

14. Aday Üretme: Birleştirme ve Budama

14.1 Sözde Kod

Hoca algoritmanın gerçek sözde kodunu (pseudocode) gösterdi ve adım adım okudu: başta veri üzerinde ilk geçiş yapılıp 1-öğe kümeleri sayılır; eşiğin altındakiler elenip F_1 bulunur; sonra $k = 2$'den başlayan döngüde aday üretme, sayma ve eleme tekrarlanır. Bu, aranan en büyük sık öğe kümesine ulaşılana kadar sürer.

```
Algorithm Apriori(T)
  C1 <- init-pass(T)                      // tek öğeleri say
  F1 <- {f | f in C1, f.count/n >= minsup}
  for (k = 2; F(k-1) != {}; k++) do
      Ck <- candidate-gen(F(k-1))         // join + prune
      for each transaction t in T do      // k. tarama
          for each candidate c in Ck do
              if c is contained in t then c.count++
      Fk <- {c in Ck | c.count/n >= minsup}
  return F <- union of all Fk
```

Soru (hocanın sorusu): Bu algoritma veritabanı üzerinde kaç geçiş yapar?

Cevap: k kez. Algoritma k-öğe sık kümelerini arıyorsa, veri kümesi üzerinde k geçiş yapar.

14.2 candidate-gen Fonksiyonu

Aday üretme fonksiyonu (candidate-gen) iki adımdan oluşur:

- **Birleştirme (join) adımı:** Gerçek sık $(k-1)$-öğe kümelerini alır, birleştirir ve bütün olası aday k-öğe kümelerini bulur.
- **Budama (prune) adımı:** Apriori'nin ikinci koşulunu, yani aşağı kapanış özelliğini sağlamayan adayları siler. Her adayın bütün alt kümelerinin sık olup olmadığına bakar. Bir alt kümesi sık değilse adayı atar.

Fonksiyon sonuç döndürdüğünde yalnız Apriori'nin ikinci özelliğini sağlayan adayları döndürür. Ardından bu adaylar işlem veri kümesi üzerinde sayılır ve desteklerinin eşiğin üzerinde olup olmadığına bakılır. Üzerinde olanlar sık k-öğe kümeleridir.

14.3 Budama Örneği

Hoca budama adımını bir örnekle tekrar gösterdi. Sık 3-öğe kümeleri:

$$F_3 = \{\{1,2,3\},\{1,2,4\},\{1,3,4\},\{1,3,5\},\{2,3,4\}\}$$

Birleştirmeden sonra slaytta şu adaylar vardı:

$$C_4 = \{\{1,2,3,4\},\{1,2,3,5\},\{1,3,4,5\}\}$$

Budama adımında her adayın bütün 3'lü alt kümelerinin F_3'te olup olmadığına bakılır:

- $\{1,2,3,4\}$: $\{1,2,3\}$, $\{2,3,4\}$, $\{1,2,4\}$, $\{1,3,4\}$ hepsi F_3'te. Kalır.
- $\{1,2,3,5\}$: $\{1,2,3\}$ var, ama $\{2,3,5\}$ yok. Atılır.
- $\{1,3,4,5\}$: $\{1,3,4\}$ var, ama $\{3,4,5\}$ yok. Atılır.

Adaylardan yalnız biri Apriori'nin ikinci özelliğini sağlar. candidate-gen fonksiyonu tek aday döndürür:

$$C_4 = \{\{1,2,3,4\}\}$$

Python ile kontrol edildi: $\{1,2,3,5\}$'in F_3'te olmayan alt kümeleri $\{1,2,5\}$ ve $\{2,3,5\}$; $\{1,3,4,5\}$'inkiler $\{1,4,5\}$ ve $\{3,4,5\}$'tir. Hocanın her aday için söylediği eksik alt küme bunlardan biridir; sonuç aynıdır.

15. Sık Öğe Kümelerinden Kural Üretme

15.1 Yöntem

Bir kural $A \to B$ biçimindedir ve "A öğe kümesini alan biri B öğe kümesini de alma eğilimindedir" diye okunur. Apriori'nin ürettiği sık öğe kümesine X diyelim. Kural üretmek için X'in bir alt kümesi A seçilip sol tarafa, X'in geri kalanı $B = X - A$ sağ tarafa konur.

- **Destek:** $A \cup B = X$ olduğu için kuralın desteği, sık öğe kümesinin desteğidir: $\text{support}(A \to B) = \text{support}(X)$. X'in sık olduğu ve desteğinin değeri zaten bilinir.
- **Güven:** Bilinmeyen tek şey güvendir. Güven, destek değerleri kullanılarak hesaplanır:

$$\text{confidence}(A \to B) = \frac{\text{support}(A \cup B)}{\text{support}(A)} = \frac{\text{support}(X)}{\text{support}(A)}$$

Payda X'in desteği, paydada sol tarafın desteği vardır.

15.2 Örnek: $\{2, 3, 4\}$

Apriori'nin ürettiği sık öğe kümesi $\{2, 3, 4\}$ olsun ve desteği %50 olsun. Bütün alt kümelerin destekleri de verilir. Altı alt küme vardır:

Alt küme	Destek
$\{2, 3\}$	%50
$\{2, 4\}$	%50
$\{3, 4\}$	%75
$\{2\}$	%75
$\{3\}$	%75
$\{4\}$	%75

Soru: Bu kümeden üretilebilecek bütün kuralları ve güven değerlerini yazın. Hoca burada işlem veri kümesinin verilmediğini vurguladı. Toplam işlem sayısını ya da sol tarafın kaç işlemde geçtiğini bilmiyoruz. Elimizde yalnız destek değerleri var. Bu yüzden destek oranıyla verilen formülü kullanmaktan başka yolumuz yok.

Önce güven hesaplamadan bütün olası kurallar yazılır: iki öğeli sol taraflar ($\{2, 3\}$, $\{2, 4\}$, $\{3, 4\}$), sonra tek öğeli sol taraflar ($\{2\}$, $\{3\}$, $\{4\}$). Sonra güvenler hesaplanır:

Kural	Hesap	Güven
2, 3 → 4	$50/50$	%100
2, 4 → 3	$50/50$	%100
3, 4 → 2	$50/75$	≈ %66,7
2 → 3, 4	$50/75$	≈ %66,7
3 → 2, 4	$50/75$	≈ %66,7
4 → 2, 3	$50/75$	≈ %66,7

Hoca ilk iki kuralı derste hesapladı, diğerlerinin de aynı yolla bulunacağını söyledi (son dört satırın hesabı eklendi).

- En küçük güven %50 ise altı kuralın hepsi yazılır.
- En küçük güven %80 ise yalnız ilk iki kural yazılır.

Değerler Python ile hesaplanıp doğrulandı.

15.3 Veriye Geri Dönmeye Gerek Yok

Bir kuralın güvenini hesaplamak için yalnız iki şey gerekir: $A \cup B$'nin, yani sık öğe kümesinin desteği ve sol tarafın desteği. Toplam işlem sayısına ya da sol tarafı içeren işlem sayısına gerek yoktur.

İlginç olan şudur: Bu bilgilerin hepsi Apriori çalışırken zaten hesaplanmıştır. Algoritma aşağıdan yukarıya ilerlerken 1-öğe, 2-öğe, 3-öğe kümelerinin destek değerlerini tek tek hesaplar. Sonunda en büyük sık öğe kümesi bulunduğunda, bütün düzeylerin destek değerlerini içeren bir tablo elde edilmiş olur. Bu değerler bir karma tabloda (hash table) tutulabilir. Böylece işlem veri kümesine geri dönüp güven hesaplamak gerekmez; tablodaki destek değerleriyle bütün olası kuralların güveni hesaplanır.

> **Hocanın vurgusu:** "Bu kısım önemli." Apriori tamamlandığında bize en çok öğe içeren sık öğe kümesini verir. Bundan yola çıkarak olabilecek bütün kurallar yazılır. Her kuralın desteği sık öğe kümesinin desteğidir. Güven için gereken bütün destek değerleri (1-öğe, 2-öğe, 3-öğe sık kümeleri) algoritma çalışırken zaten hesaplanıp kaydedilmiştir. Güven hesabı için veritabanına dönmeye gerek yoktur.

16. Pratikte Apriori: k Sınırı ve Maliyet

16.1 k'nin Üst Sınırı

Apriori düzey düzey, aşağıdan yukarıya arama yaparak en büyük k boyutlu sık öğe kümesini bulur. Bunun için veri üzerinde k geçiş yapar. Veri kümesi çok büyük olabilir. Hocanın örneği: Ülke çapındaki bütün mağazaları olan bir zincir marketin son altı aylık işlem verisi. Bu kadar büyük veri üzerinde çok sayıda geçiş, algoritmanın çok uzun sürmesi demektir.

Bu yüzden pratikte k sınırlandırılır. En büyük sık öğe kümesinin boyutu bir üst değerle bağlanır; hocaya göre bu değer genellikle 10'dur.

Hoca bunun mantıklı olduğunu da açıkladı. Bu işi zaten birlikte satılan popüler öğeleri bulmak için yapıyoruz. Genellikle birlikte satılan 2, 3 ya da 4 öğeyi bulmak yeterlidir. On öğenin birlikte satıldığını bulmak pratikte pek ilginç değildir. Birlikte satılan iki üç ürün bulunursa şunlar yapılabilir:

- Ürünler fiziksel mağazada yan yana konur, insanlar daha kolay alsın diye.
- Ya da ürünler mağazanın çok uzak noktalarına konur, araya satılmak istenen başka ürünler yerleştirilir. Müşteri bu ürünleri zaten alacaktır; alırken mağazadaki diğer ürünleri de görsün, dolaşsın diye.
- Ya da popüler olarak birlikte satılan üç ürünün yanına çok daha az satılan dördüncü bir ürün eklenip çapraz satış yapılır.

16.2 Asıl Maliyet: Disk Erişimi

> **Hocanın vurgusu:** Algoritmanın kendisi hızlı çalışır. Ama her yinelemesinde veritabanına bir disk erişimi (I/O access) vardır. I/O erişimi, bellek işlemine göre maliyeti yüksek bir işlemdir. Her erişimde veritabanındaki bütün işlemler sırayla (sequential) okunmak zorunda olduğu için asıl zamanı alan kısım bu okumadır.

17. Veri Biçimleri

Apriori ve benzeri algoritmaların görmek istediği biçim, her satırın bir işlemi ifade ettiği biçimdir. Satırdaki değerler, o işlemde satılan öğelerdir.

Ama işlem veritabanı bazen tablo biçiminde, yani nitelik-değer çiftleri hâlinde olabilir. Bu durumda veri önce işlem biçimine dönüştürülür. Slayttaki örnekte her nitelik-değer çifti bir öğe yapılmıştı: `(Attribute1, a)`, `(Attribute2, b)`, `(Attribute3, d)` gibi. Bu dönüşümden sonra veri Apriori ve benzeri algoritmalara girdi olarak verilebilir.

Ders Sonrası Notlar

> **Ders sonrası not:** Slayttaki güven formülünün koşullu olasılık gösterimi hatalıydı. Arada bir öğrenci, payda hem X'in hem Y'nin bulunması gerektiğini belirtti; hoca haklı olduğunu söyledi ve slaytı düzelteceğini belirtti. Doğrusu: $\text{conf}(X \to Y) = P(Y \mid X) = \text{count}(X \cup Y)/\text{count}(X)$.
>
> **Ders sonrası not:** Hoca 01:51:02 ve 01:54:10 civarında, birleşimin desteği için iki kez "denominator" (payda) dedi. Doğrusu: $\text{support}(A \cup B)$ paydadır (numerator); paydada sol tarafın desteği bulunur.
>
> **Ders sonrası not:** Hoca 00:46:05'te "ikiden fazla öğe içeren kümeler için kural yazabilirim" dedi. Doğrusu: en az iki öğe içeren kümelerden kural yazılır; nitekim örnekte iki öğeli $\{A, D\}$ kullanıldı. Aynı şekilde 00:48:29'daki "bütün kuralları yazdıktan sonra destek değerlerini hesaplıyorum" ifadesindeki ölçü güvendir; destek iki kural için zaten aynıdır.
>
> **Ders sonrası not:** Hoca 01:25:08'de bir an özyinelemeli (recursive) algoritmadan söz etti. Apriori özyinelemeli değildir; döngüyle çalışan yinelemeli bir algoritmadır.
>
> **Ders sonrası not:** 01:34:26'daki "%50, yani at least one item" ifadesi sürçmedir. Hocanın hemen ardından söylediği gibi, dört işlemde %50, öğe kümesinin en az 2 işlemde geçmesi demektir.

Ders sonrası not: Hoca sık öğe kümelerini zaman zaman "en büyük öğe kümeleri" diye tanımladı ve kuralları yalnız en büyük sık öğe kümesinden yazdı. Apriori'nin 1. adımı desteği eşiği geçen bütün öğe kümelerini bulur; kural üretimi de iki ya da daha çok öğeli her sık öğe kümesine uygulanır. Örnek 2'de $\{$Chicken, Clothes, Milk$\}$'in iki öğeli sık alt kümelerinden de minconf'u geçen kurallar çıkabilir; bunlar yalnız en büyük kümeye bakılırsa kaçırılır.

Ders sonrası not: Hoca $\{1,2,3\}$ ve $\{1,3,5\}$ gibi adayların birleştirmede üretilip budamada atıldığını anlattı. Ders kitabındaki candidate-gen büyük olasılıkla yalnız ilk $k-2$ öğesi aynı olan iki kümeyi birleştirir. Python denetimine göre bu kuralla C_3'te yalnız $\{2,3,5\}$ üretilir; ikinci örnekte de $\{1,2,3,5\}$ hiç üretilmez, $\{1,2,3,4\}$ ve $\{1,3,4,5\}$ üretilir. Sonuç ve budama mantığı aynıdır; fark yalnız ara adaylardadır.

Ders sonrası not: "Genellikle 10" değeri hocanın pratik önerisidir. Algoritmanın kendisinde böyle bir sınır yoktur; aday kalmayınca (F_{k-1} boş olunca) durur.

Örnek Sorular

Soru 1 (derste sözlü, 00:43:34) Beş işlemlik Örnek 1 verisinden (minsup = %50, minconf = %50) ilişkilendirme kuralları üretebilir misiniz?

Çözüm (derste verildi): Sık örüntüler $\{A{:}3, B{:}3, D{:}4, E{:}3, AD{:}3\}$. Kural yalnız $\{A, D\}$'den çıkar. $A \to D$: destek %60, güven $3/3 = \%100$. $D \to A$: destek %60, güven $3/4 = \%75$. İkisi de eşikleri geçer.

Soru 2 (derste sözlü, 00:50:36) Örnek 1'de $A \to D$ kuralının desteği neden %80 değil de %60'tır?

Çözüm (derste verildi): Destek, A ile D'yi birlikte içeren işlemlerin oranıdır, A veya D içerenlerin değil. Birlikte içeren işlem 3, toplam 5: $3/5 = \%60$.

Soru 3 (derste sözlü, 00:57:34) Örnek 2'de (minconf = %80) Chicken, Milk → Clothes kuralı alınır mı?

Çözüm (derste verildi): Chicken ve Milk birlikte 4 işlemde geçer, bunların 3'ünde Clothes da vardır. Güven $3/4 = \%75 < \%80$, kural alınmaz.

Soru 4 (derste sözlü, 01:42:29) Apriori algoritması veritabanı üzerinde kaç geçiş yapar?

Çözüm (derste verildi): k-öğe sık kümeleri aranıyorsa k geçiş.

Soru 5 (derste sözlü, 01:52:34) Desteği %50 olan sık öğe kümesi $\{2,3,4\}$ ve alt kümelerinin destekleri ($\{2,3\}$: %50, $\{2,4\}$: %50, $\{3,4\}$: %75, $\{2\}$, $\{3\}$, $\{4\}$: %75) veriliyor. Üretilebilecek bütün kuralları ve güvenlerini yazın. minconf %50 ve %80 için hangi kurallar alınır?

Çözüm (derste verildi): $2,3 \to 4$ ve $2,4 \to 3$: %100. $3,4 \to 2$, $2 \to 3,4$, $3 \to 2,4$, $4 \to 2,3$: $50/75 \approx \%66{,}7$. minconf %50'de altısı, %80'de yalnız ilk ikisi alınır. (Son dört kuralın sayısal değeri derste söylenmedi, python ile hesaplandı.)

Dersin Toparlaması

Bu derste ilişkilendirme kuralı madenciliğinin perakendedeki çapraz satış ve üst satış ihtiyacından doğduğu, Agrawal'ın market sepeti analizi olarak tanımladığı problemin daha sonra e-ticarete ve webdeki belge kümelerinde birlikte geçen anahtar sözcükleri bulmaya taşındığı görüldü. Bir kuralın $X \to Y$ biçiminde, kesişimi boş iki öğe kümesinden oluştuğu; desteğin iki tarafı birlikte içeren işlemlerin oranı, güvenin ise sol tarafı içeren işlemler içinde sağ tarafı da içerenlerin oranı olduğu iki örnek üzerinde hesaplandı. Desteğin kuralın yönüne bağlı olmadığı, güvenin ise bağlı olduğu vurgulandı.

Ardından Apriori algoritması ayrıntılı işlendi: aşağı kapanış özelliği, aşağıdan yukarıya düzey düzey arama, birleştirme ve budama adımlarından oluşan aday üretme, k geçiş ve kural üretiminde güvenin veriye dönmeden, algoritma sırasında kaydedilen destek değerleriyle hesaplanması. Pratikte k'nin sınırlandırıldığı ve asıl maliyetin her geçişteki disk okuması olduğu belirtildi. Sıralı örüntü madenciliği gelecek derse bırakıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.